Université de Rouen Normandie — M1 GIL

Architecture Logicielle 2025–2026

Mini-projet : Outils de dessin vectoriel

Rapport de conception et d'implémentation

Auteur Ahcène AMOUCHAS

Encadrant F. Nicart

Date Mai 2026

À rendre avant le 15 mai 2026

0 Table des matières

1 Introduction **2**

2 Architecture globale **3**
2.1 Vue d'ensemble . . . 3
2.2 Description des paquetages . . . 3
2.3 Flux de données . . . 4

3 Patrons de conception **5**
3.1 Composite . . . 5
3.2 Visiteur . . . 6
3.3 Fabrique (Factory Method) . . . 7
3.4 Monteur (Builder) . . . 8
3.5 Façade . . . 9
3.6 Observateur . . . 9

4 Patrons envisagés mais écartés **11**

5 Format de sauvegarde XML **12**
5.1 Choix de conception . . . 12
5.2 Schéma XSD . . . 12
5.3 Exemple de fichier `.vec` . . . 12

6 Bilan du respect des principes SOLID **14**
6.1 Responsabilité unique (SRP) . . . 14
6.2 Ouvert/Fermé (OCP) . . . 14
6.3 Substitution de Liskov (LSP) . . . 15
6.4 Ségrégation des interfaces (ISP) . . . 15
6.5 Inversion des dépendances (DIP) . . . 15

A Liste complète des entités par paquetage **16**

1 Introduction

Ce rapport présente la conception et l'implémentation d'un ensemble d'outils de création et de manipulation de dessins vectoriels, réalisé dans le cadre du mini-projet d'architecture logicielle du M1 GIL.

L'objectif principal n'est pas de produire un logiciel complet et ergonomique, mais de mettre en œuvre une architecture de qualité, fondée sur les grands principes vus en cours : modularité, séparation des responsabilités, et utilisation raisonnée des patrons de conception.

Le projet comprend trois programmes distincts :

— `edit` — un éditeur interactif semi-visuel,

— `merge` — un utilitaire de fusion de deux fichiers vectoriels,

— `v2bmp` — un convertisseur vers le format PNG.

Les sections suivantes décrivent l'architecture globale, les patrons de conception mis en œuvre, le format de sauvegarde XML, et un bilan du respect des principes SOLID.

2 Architecture globale

2.1 Vue d'ensemble

L'application est organisée en sept paquetages distincts, chacun portant une responsabilité précise et cohérente.

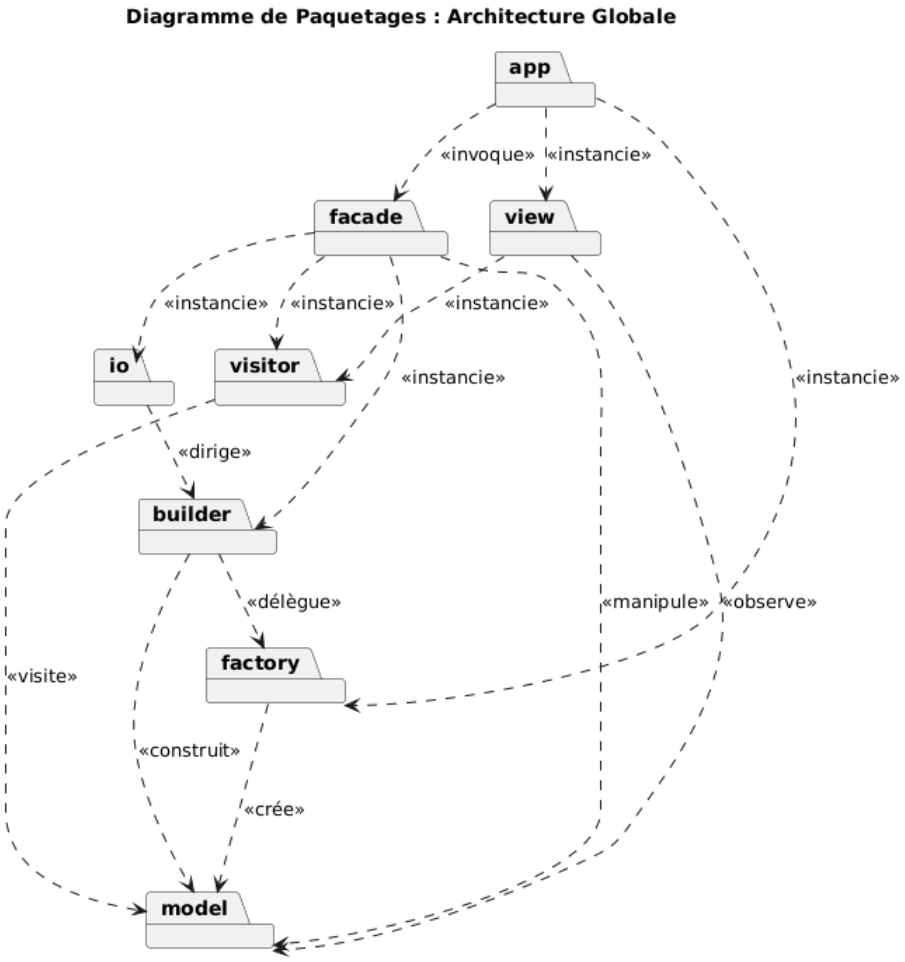

Figure 1 – Diagramme de paquetages de l'application

2.2 Description des paquetages

`app` Point d'entrée des trois programmes (`Edit`, `Merge`, `V2Bmp`). Ces classes contiennent uniquement la méthode `main` et délèguent tout le traitement à la façade.

`model` Cœur du domaine métier. Contient la hiérarchie Composite (`Shape`, `Line`, `Rectangle`, `Circle`, `Ellipse`, `Group`), la classe `Drawing` ainsi que l'interface `DrawingObserver` du patron Observateur.

`factory` Implémentation du patron Fabrique. Contient l'interface `ShapeCreator`, les créateurs concrets pour chaque forme, et le registre `CommandRegistry` qui centralise la création.

builder Implémentation du patron Monteur. Contient l'interface `DrawingBuilder` et sa réalisation `CompositeDrawingBuilder`, responsable de l'assemblage de l'arbre Composite en mémoire.

io Entrées/sorties fichiers. Contient le `XmlDrawingDirector`, qui orchestre la lecture XML en coordonnant le Monteur et la Fabrique.

visitor Implémentation du patron Visiteur. Contient l'interface `ShapeVisitor` et ses visiteurs concrets : `ListVisitor`, `XmlSaveVisitor` et `GraphicsVisitor`.

facade Implémentation du patron Façade. La classe `DrawingFacade` expose une interface simple aux trois programmes principaux, masquant la complexité du sous-système.

view Interface graphique. Contient `DrawingView`, la fenêtre Swing qui implémente `DrawingObserver` et se rafraîchit automatiquement à chaque modification du dessin.

2.3 Flux de données

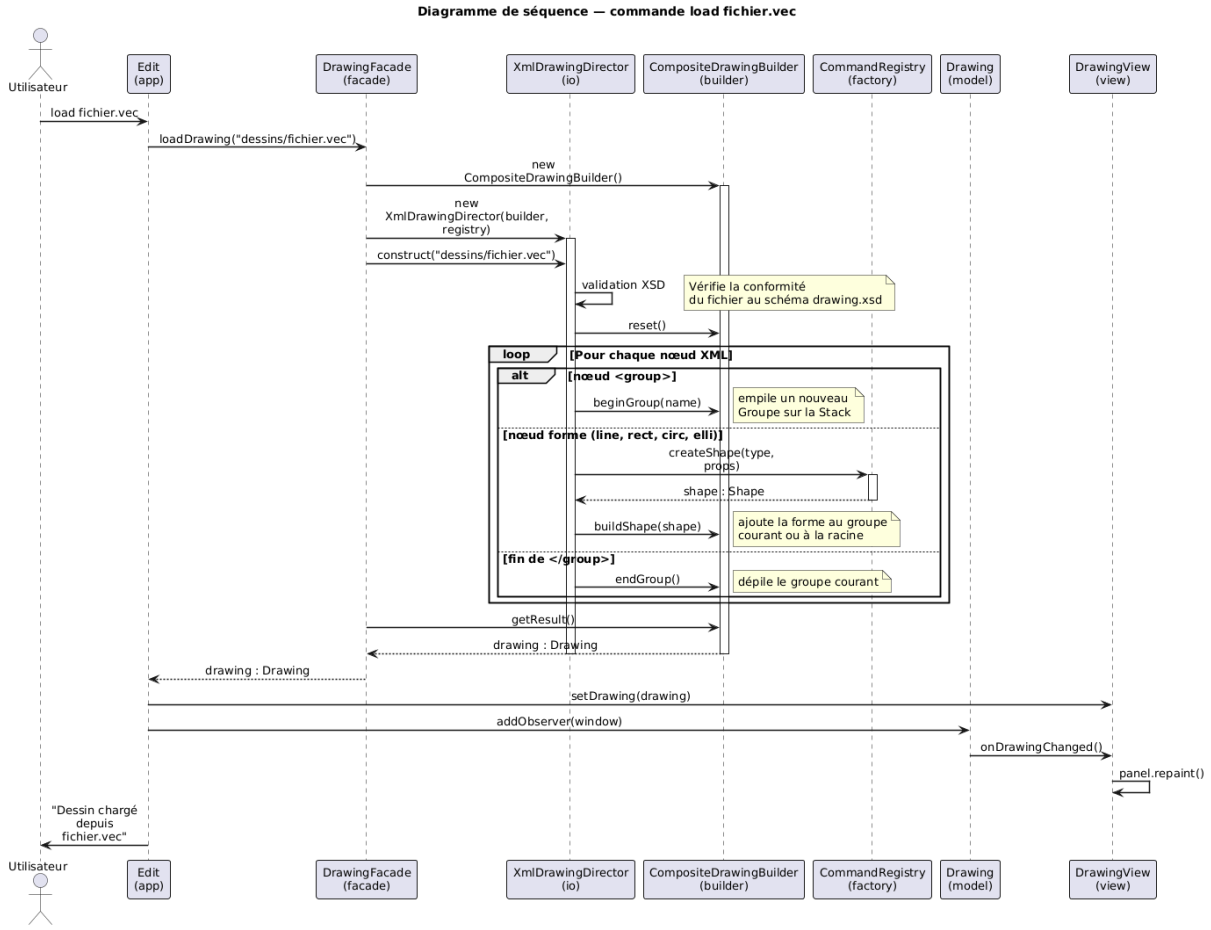

Figure 2 – Exemple de flux : chargement d'un fichier `.vec`

3 Patrons de conception

3.1 Composite

Motivation

Un dessin vectoriel est composé de formes primitives (ligne, rectangle, cercle, ellipse) et de groupes pouvant contenir d'autres formes ou d'autres groupes. Le patron Composite permet de traiter uniformément un élément simple et un groupe imbriqué, sans que le code client ait besoin de distinguer les deux cas.

Mise en œuvre

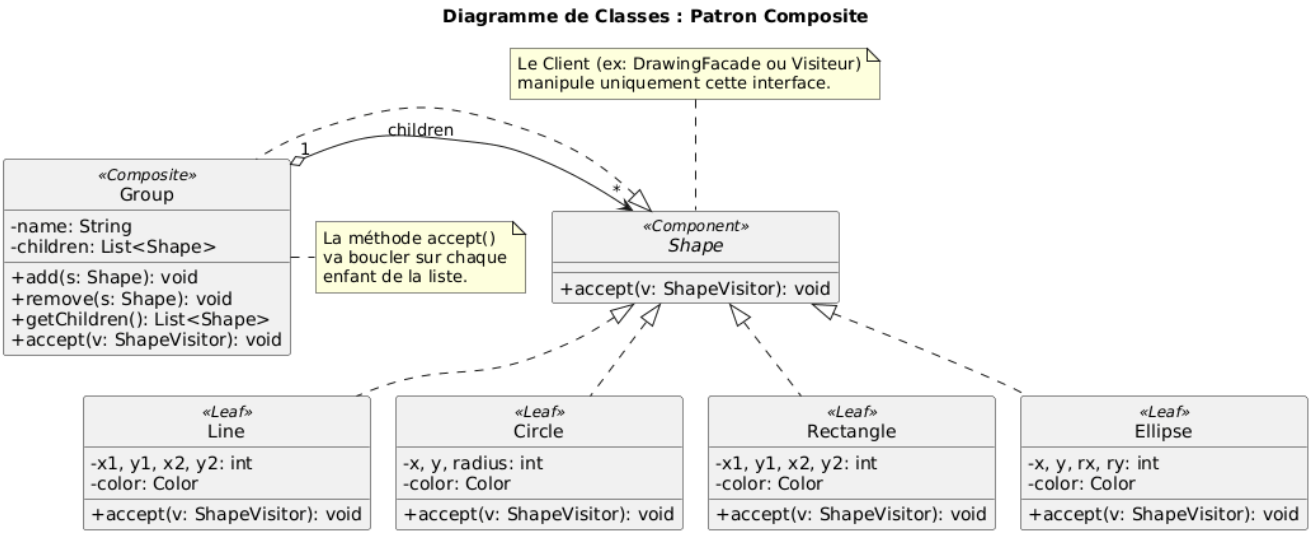

Figure 3 – Patron Composite instancié dans l'application

Rôle dans le patron	Entité dans l'application
Component	`model.Shape`
Leaf	`Line`, `Rectangle`, `Circle`, `Ellipse`
Composite	`model.Group`
Client	`Drawing`, visiteurs, `DrawingFacade`

3.2 Visiteur

Motivation

Les trois programmes (`edit`, `merge`, `v2bmp`) appliquent des traitements différents sur la même structure Composite. Sans le patron Visiteur, il faudrait modifier chaque classe de forme à chaque nouvel opération, violant le principe Ouvert/Fermé. Le Visiteur externalise ces opérations dans des classes dédiées, laissant le modèle inchangé.

Mise en œuvre

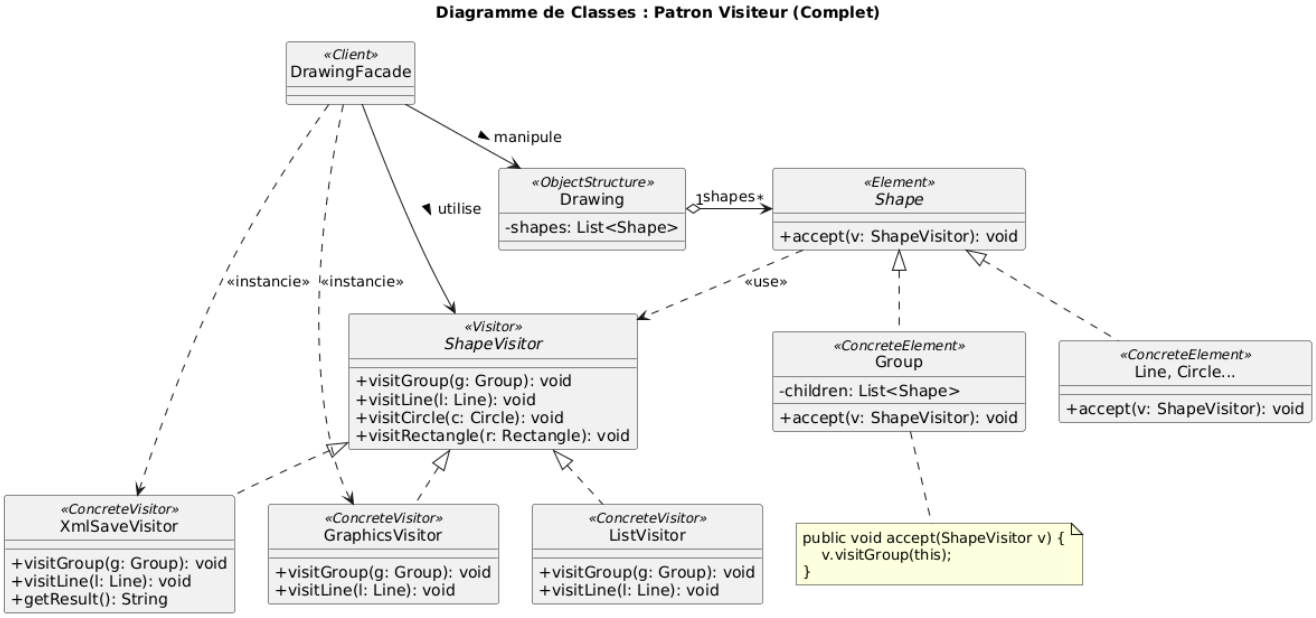

Figure 4 – Patron Visiteur instancié dans l'application

Rôle dans le patron	Entité dans l'application
Visitor	`visitor.ShapeVisitor`
ConcreteVisitor	`ListVisitor`, `XmlSaveVisitor`, `GraphicsVisitor`
Element	`model.Shape`
ConcreteElement	`Line`, `Rectangle`, `Circle`, `Ellipse`, `Group`
ObjectStructure	`model.Drawing`

3.3 Fabrique (Factory Method)

Motivation

L'interpréteur de commandes et le parser XML doivent instancier des formes sans connaître leurs classes concrètes. La Fabrique délègue cette responsabilité à des créateurs spécialisés, réduisant le couplage et facilitant l'ajout d'un nouveau type de forme.

Mise en œuvre

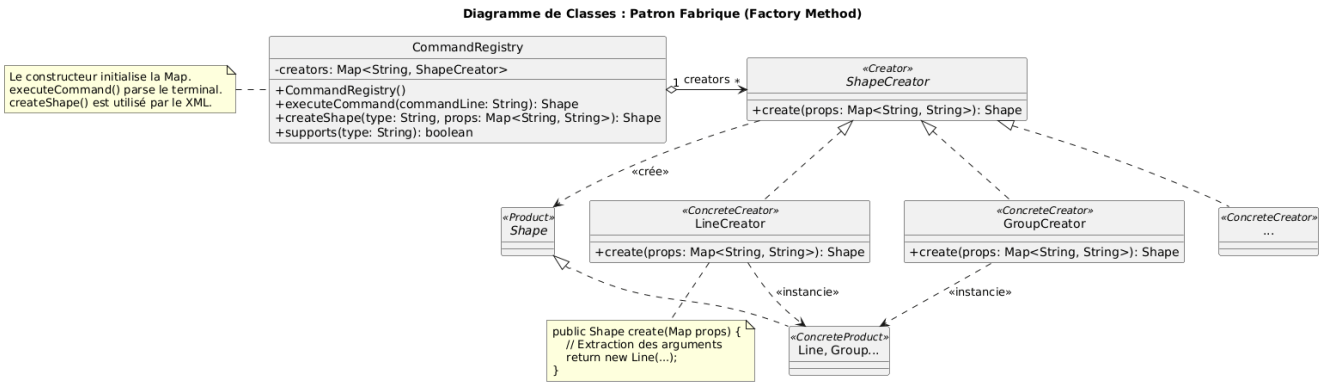

Figure 5 – Patron Fabrique instancié dans l'application

Rôle dans le patron	Entité dans l'application
Creator	`factory.ShapeCreator`
ConcreteCreator	`LineCreator`, `RectangleCreator`, `CircleCreator`, ...
Product	`model.Shape`
ConcreteProduct	`Line,Rectangle,Circle...`

Collaboration Factory/Builder

La Fabrique et le Monteur collaborent lors du chargement d'un fichier : le `XmlDrawingDirector` utilise la Fabrique pour créer chaque forme individuelle, puis le Monteur pour l'assembler au bon endroit dans l'arbre.

3.4 Monteur (Builder)

Motivation

La construction d'un dessin depuis un fichier XML est un processus multi-étapes nécessitant la gestion de l'imbrication des groupes. Le patron Monteur sépare la logique d'assemblage (`CompositeDrawingBuilder`) de la logique d'orchestration (`XmlDrawingDirector`), rendant chacun indépendant et testable séparément.

Mise en œuvre

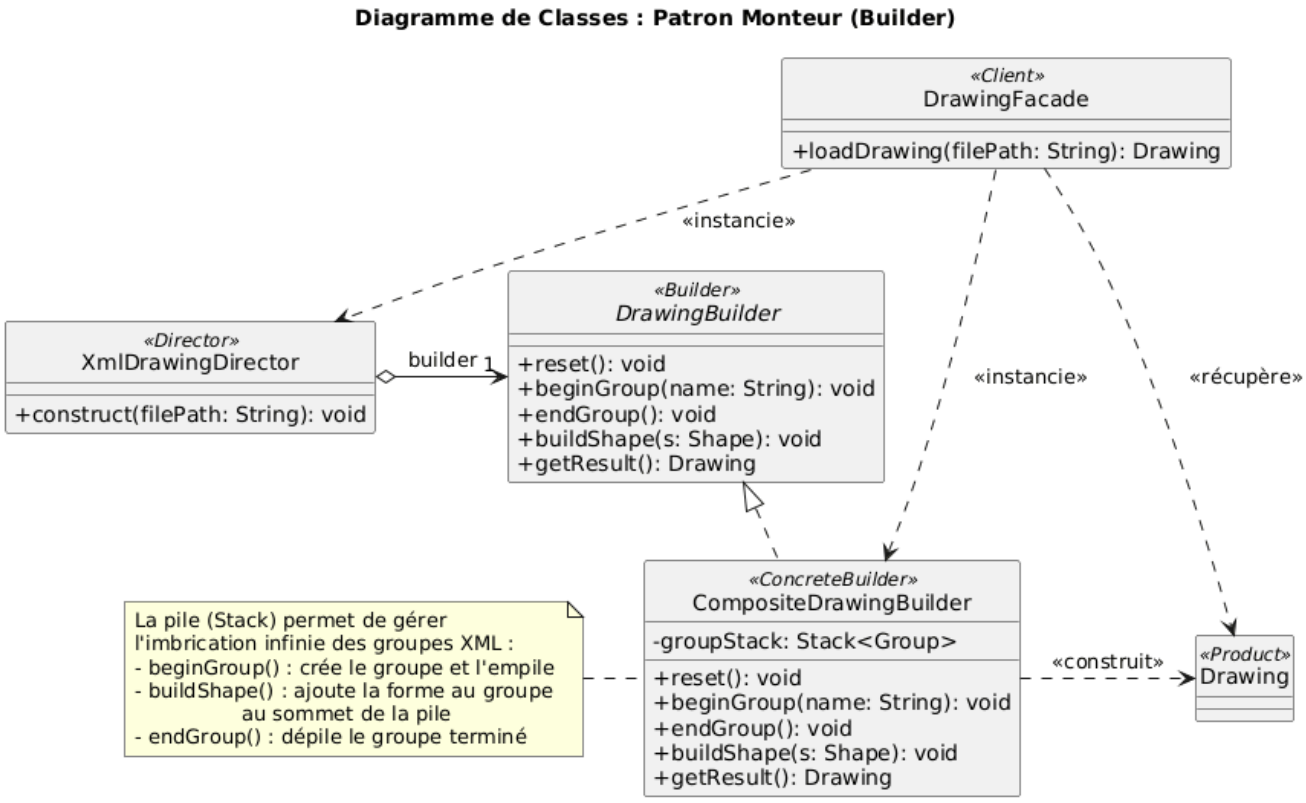

Figure 6 – Patron Builder instancié dans l'application

Rôle dans le patron	Entité dans l'application
Builder	`builder.DrawingBuilder`
ConcreteBuilder	`builder.CompositeDrawingBuilder`
Director	`io.XmlDrawingDirector`
Product	`model.Drawing`

Gestion des groupes imbriqués

Une pile (`Stack<Group>`) maintient le contexte d'imbrication courant. `beginGroup()` empile un nouveau groupe ; `endGroup()` le dépile. Toute forme ajoutée via `buildShape()`

est rattachée au groupe au sommet de la pile, ou à la racine du dessin si la pile est vide.

3.5 Façade

Motivation

Les trois programmes principaux (`Edit`, `Merge`, `V2Bmp`) doivent accéder à un sous-système complexe impliquant le modèle, la sérialisation XML, la conversion bitmap et la gestion des fichiers. La Façade expose une interface simple et stable, masquant ces détails et rendant les `main` minimalistes.

Mise en œuvre

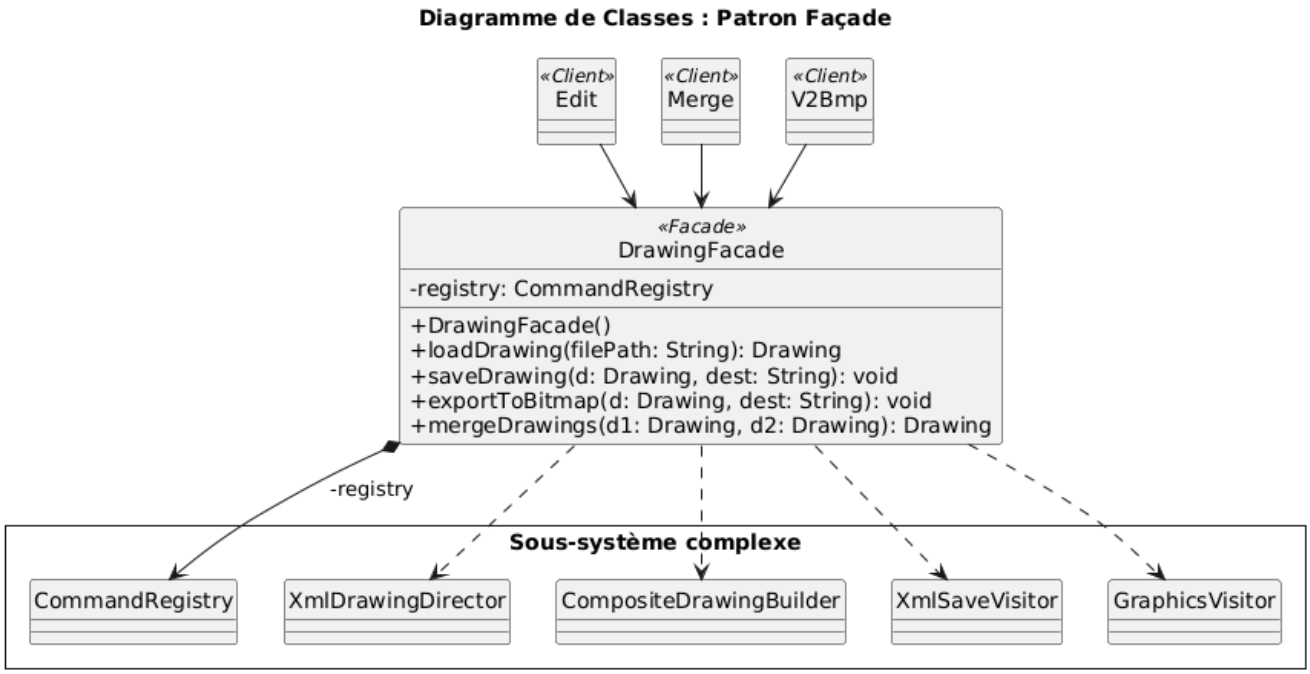

Figure 7 – Patron Façade instancié dans l'application

Rôle dans le patron	Entité dans l'application
Façade	`facade.DrawingFacade`
Client	`app.Edit`, `app.Merge`, `app.V2Bmp`
Sous-système	`io`, `builder`, `visitor`, `model`

3.6 Observateur

Motivation

La fenêtre graphique de l'éditeur doit se rafraîchir automatiquement à chaque modification du dessin (ajout, suppression, groupement). Le patron Observateur découple

le modèle (`Drawing`) de la vue (`DrawingView`) : le modèle notifie ses observateurs sans connaître leur type concret.

Mise en œuvre

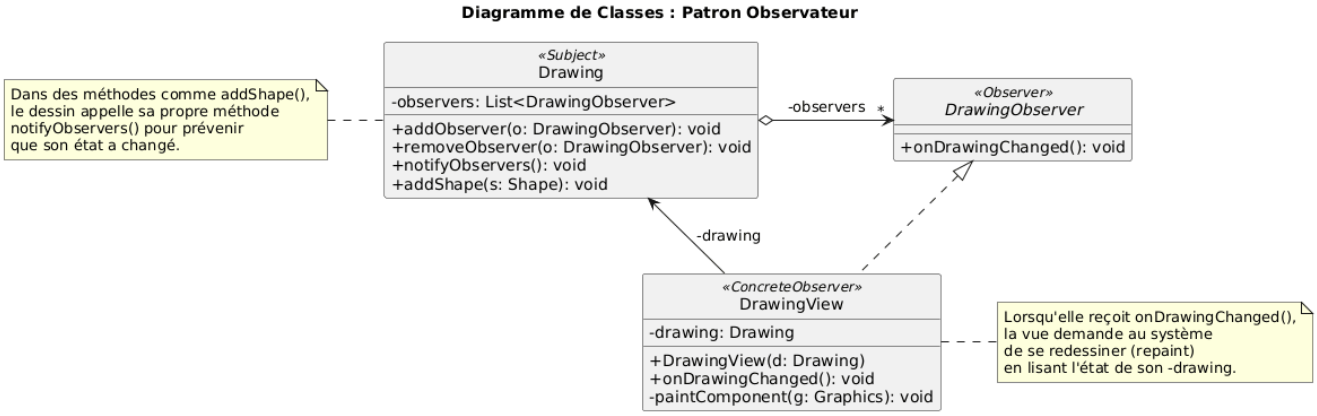

Figure 8 – Patron Observateur instancié dans l'application

Rôle dans le patron	Entité dans l'application
Subject	`model.Drawing`
Observer	`model.DrawingObserver`
ConcreteObserver	`view.DrawingView`

Note de conception

L'interface `DrawingObserver` a été maintenue dans le paquetage `model` par cohésion avec `Drawing`. La déplacer dans `view` aurait introduit une dépendance inverse `model` → `view`, violant le principe MVC. Un paquetage `observer` dédié aurait été envisageable mais disproportionné pour une seule interface.

4 Patrons envisagés mais écartés

Lors de la phase de conception, j'ai évalué d'autres patrons de conception avant de retenir mon architecture finale :

Fabrique Abstraite (*Abstract Factory*)
Au début, je me suis demandé s'il fallait utiliser une Abstract Factory au lieu d'une simple Factory Method. Cependant, comme le sujet ne définit qu'une seule et unique famille de formes, ce patron aurait nécessité de créer de nombreuses interfaces supplémentaires pour rien. J'ai préféré éviter cette sur-ingénierie.

Proxy J'ai pensé à utiliser un Proxy virtuel pour optimiser la mémoire (chargement paresseux / *lazy loading* du fichier XML uniquement au moment de l'affichage). Finalement, vu la taille modeste des fichiers manipulés dans le cadre de ce projet, cette solution m'a paru disproportionnée et a été écartée.

Pont (*Bridge*)
J'ai hésité entre le Pont et le Visiteur pour séparer la logique des formes de celle des rendus (écran, export XML). Le Visiteur m'a semblé beaucoup plus adapté et élégant, car il permet d'ajouter de nouveaux comportements sans avoir à créer une double hiérarchie d'héritage complexe comme l'aurait imposé le Pont.

Singleton
J'ai pensé au Singleton pour le `CommandRegistry` car il n'y en a qu'un seul par exécution. Mais comme ce registre n'est utilisé que par la `DrawingFacade`, il m'a suffi de l'instancier au démarrage de l'application (dans le `main`) et de le lui passer en paramètre. Créer un Singleton global n'était donc ni nécessaire, ni utile.

5 Format de sauvegarde XML

5.1 Choix de conception

Les dessins sont sauvegardés au format XML avec l'extension `.vec`. La structure du fichier reflète directement l'arbre Composite : les groupes sont des éléments imbriqués, et les formes primitives sont des éléments feuilles avec leurs attributs.

5.2 Schéma XSD

La validité des fichiers `.vec` est vérifiée à chaque chargement grâce à un schéma XSD (`drawing.xsd`).

```
<xs:element name="drawing" type="shapeContainer"/>

<xs:complexType name="shapeContainer">
  <xs:choice minOccurs="0" maxOccurs="unbounded">
    <xs:element name="line">
      <xs:complexType>
        <xs:attribute name="x1"    type="xs:int"     use="required"/>
        <xs:attribute name="y1"    type="xs:int"     use="required"/>
        <xs:attribute name="x2"    type="xs:int"     use="required"/>
        <xs:attribute name="y2"    type="xs:int"     use="required"/>
        <xs:attribute name="color" type="colorType"  use="required"/>
      </xs:complexType>
    </xs:element>
    <!-- idem pour rect, circ, elli -->
    <xs:element name="group">
      <xs:complexType>
        <xs:complexContent>
          <xs:extension base="shapeContainer">
            <xs:attribute name="name" type="xs:string" use="required"/>
          </xs:extension>
        </xs:complexContent>
      </xs:complexType>
    </xs:element>
  </xs:choice>
</xs:complexType>
```

Listing 1 – Extrait du schéma XSD (drawing.xsd)

5.3 Exemple de fichier `.vec`

```
<?xml version="1.0" encoding="UTF-8"?>
<drawing>
  <line x1="10" y1="5" x2="42" y2="23" color="blue"/>
```

```
  <group name="maison">
    <rect x1="100" y1="200" x2="300" y2="400" color="red"/>
    <circ x="200" y="150" radius="50" color="green"/>
  </group>
</drawing>
```

Listing 2 – Exemple de fichier vectoriel sauvegardé

6 Bilan du respect des principes SOLID

6.1 Responsabilité unique (SRP)

La majorité des classes respectent strictement le SRP. Ainsi, `XmlSaveVisitor` gère uniquement la sérialisation XML ; `CompositeDrawingBuilder` gère uniquement l'assemblage de l'arbre Composite en mémoire ; chaque créateur concret (`LineCreator`, `CircleCreator`, etc.) n'instancie qu'un seul type de forme ; et `XmlDrawingDirector` orchestre uniquement la lecture XML sans jamais construire lui-même.

Deux classes présentent néanmoins des responsabilités élargies, assumées comme compromis de conception :

`Edit` — compromis d'architecture justifié. La classe `Edit` concentre la boucle principale d'interprétation, la gestion de l'état courant (dessin en cours, chemin des fichiers) et le branchement de la vue. Une décomposition stricte aurait conduit à un `CommandInterpreter` dédié au *dispatch* des commandes et une classe `ApplicationState` portant l'état mutable. Ce choix n'a pas été retenu afin de ne pas multiplier les classes pour un interpréteur dont la logique reste linéaire et lisible dans un seul `switch`. Ce compromis est documenté et n'affecte pas les autres composants du système.

`CommandRegistry` — couplage parsing/délégation. `CommandRegistry` assure deux rôles : la traduction d'une ligne de commande textuelle en dictionnaire de propriétés (`Map`), et la délégation de la création au bon `ShapeCreator`. Une séparation stricte aurait produit un `CommandParser` ne contenant qu'une seule méthode, ce qui aurait constitué une sur-ingénierie disproportionnée. La cohésion fonctionnelle des deux rôles — tous deux liés à l'interprétation d'une commande — justifie leur regroupement.

6.2 Ouvert/Fermé (OCP)

L'ajout d'un nouveau type de forme ne nécessite que :

1. la création d'une classe concrète implémentant `Shape`,
2. l'ajout d'une méthode dans `ShapeVisitor` et son implémentation dans chaque visiteur concret,
3. l'enregistrement d'un `ShapeCreator` dans `CommandRegistry`.

Aucune classe existante n'est modifiée.

Violation localisée — `CommandRegistry.executeCommand()`. La méthode `executeCommand()` contient un `switch` sur le type de forme pour traduire les arguments textuels en dictionnaire de propriétés. L'ajout d'une nouvelle forme impose de modifier cette méthode, ce qui viole l'OCP. J'avais envisagé de déléguer ce parsing à chaque `ShapeCreator`, mais cela

lui aurait donné une seconde responsabilité — la création *et* le parsing — violant à son tour le SRP. Face à ce conflit, j'ai privilégié le SRP. La violation reste contenue dans une seule méthode et son impact se limite à l'ajout d'un `case` supplémentaire.

6.3 Substitution de Liskov (LSP)

Toutes les formes concrètes implémentent `Shape` et peuvent être substituées sans altérer le comportement du programme. En particulier, `Group` est utilisé partout où un `Shape` est attendu, et les visiteurs le traitent de façon homogène via `accept()`.

Violation localisée — `Group.getColor()`. `Group` retourne `null` dans `getColor()`, car un groupe ne possède pas de couleur propre. Cela viole le LSP : un client manipulant un `Shape` générique risque une `NullPointerException` en appelant `getColor()` sur un groupe. J'avais envisagé de faire retourner `Optional<Color >` à l'interface `Shape`, mais cette modification aurait entraîné l'adaptation de tous les visiteurs concrets. Par manque de temps, je ne l'ai pas appliquée. Le risque reste toutefois maîtrisé : aucun visiteur actuel n'appelle `getColor()` sur un `Group`.

6.4 Ségrégation des interfaces (ISP)

Les interfaces sont fines et ciblées : `ShapeCreator` n'expose qu'une méthode, `DrawingObserver` n'en expose qu'une, `DrawingBuilder` définit exactement les opérations nécessaires au montage. `ShapeVisitor` impose l'implémentation de toutes les formes à chaque visiteur, ce qui est une légère violation de l'ISP, mais acceptable car tous les visiteurs traitent effectivement l'ensemble des formes.

6.5 Inversion des dépendances (DIP)

Les dépendances vont toujours vers les abstractions : `XmlDrawingDirector` dépend de `DrawingBuilder` (interface) et non de `CompositeDrawingBuilder` (concret) ; `CommandRegistry` manipule des `ShapeCreator` et des `Shape`, jamais leurs implémentations directement.

Instanciation dans le `main`. Pour respecter le DIP, les dépendances sont injectées par constructeur (comme le `CommandRegistry` dans `DrawingFacade`). L'instanciation des objets concrets est strictement isolée aux points d'entrée du programme (`Edit`, `Merge`, `V2Bmp`). Ce n'est pas une violation du principe, mais une nécessité : le `main` doit assembler les objets au démarrage pour que tout le reste de l'application puisse rester parfaitement découplé.

A Liste complète des entités par paquetage

A.0 Paquetage `app`

Entité	Responsabilité
`Edit`	Point d'entrée de l'éditeur interactif
`Merge`	Point d'entrée du programme de fusion
`V2Bmp`	Point d'entrée du convertisseur PNG

A.0 Paquetage `model`

Entité	Responsabilité
`Shape`	Interface commune à tous les éléments du dessin
`Line`	Représente une ligne entre deux points
`Rectangle`	Représente un rectangle défini par deux coins
`Circle`	Représente un cercle défini par un centre et un rayon
`Ellipse`	Représente une ellipse à rayons horizontal et vertical
`Group`	Regroupe plusieurs formes en un élément composite
`Drawing`	Contient la liste racine des formes et gère les observateurs
`Color`	Énumération des couleurs symboliques supportées
`DrawingObserver`	Interface du patron Observateur pour la notification des vues

A.0 Paquetage `factory`

Entité	Responsabilité
`ShapeCreator`	Interface de la fabrique (Factory Method)
`LineCreator`	Crée des instances de `Line`
`RectangleCreator`	Crée des instances de `Rectangle`
`CircleCreator`	Crée des instances de `Circle`
`EllipseCreator`	Crée des instances de `Ellipse`
`GroupCreator`	Crée des instances de `Group`
`CommandRegistry`	Registre central associant un type à son créateur

A.0 Paquetage `builder`

Entité	Responsabilité
`DrawingBuilder`	Interface du monteur
`CompositeDrawingBuilder`	Assemble l'arbre Composite en mémoire via une pile

A.0 Paquetage `io`

Entité	Responsabilité
`XmlDrawingDirector`	Orchestre la lecture XML en pilotant le monteur et la fabrique

A.0 Paquetage `visitor`

Entité	Responsabilité
`ShapeVisitor`	Interface du visiteur, définit une méthode par type de forme
`ListVisitor`	Affiche la liste numérotée des formes dans la console
`XmlSaveVisitor`	Génère la représentation XML du dessin
`GraphicsVisitor`	Dessine les formes sur un contexte graphique Java 2D

A.0 Paquetage `facade`

Entité	Responsabilité
`DrawingFacade`	Expose les opérations de haut niveau aux programmes principaux

A.0 Paquetage `view`

Entité	Responsabilité
`DrawingView`	Fenêtre graphique Swing, observateur du dessin courant